

Python 최적화 과제 보고서

YOLOv8 실험 관리 코드 구조 개선

1. 대상 코드 소개

본 과제에서는 ultralytics YOLOv8을 사용하는 객체 탐지 실험 스크립트를 대상으로 삼았다. 해당 코드는 모델 학습(train)과 평가(val)를 반복적으로 실행하면서 결과를 저장하고 기록하는 용도로 쓰이는데, 실험 조건을 조금씩 바꿔가며 여러 번 돌려야 하는 특성상 설정 관리와 코드 재사용이 중요하다. 이 코드를 선택한 이유는 문제점이 뚜렷하면서도 수업에서 다룬 개념을 자연스럽게 적용할 수 있는 구조였기 때문이다.

2. 기존 코드의 문제점

기존 코드(src/before/run_experiment.py)에서 확인된 주요 문제는 크게 세 가지였다.

첫째, 설정값이 함수 안에 하드코딩되어 있었다. model_name, epochs, lr 등 열 개가 넘는 값이 run_experiment() 내부에 지역변수로 박혀 있어서, 실험 조건을 하나 바꾸려면 함수 안으로 들어가 직접 수정해야 했다.

둘째, 하나의 함수가 너무 많은 일을 하고 있었다. 모델 로딩, 학습, 평가, 로그 파일 작성, JSON 저장이 run_experiment() 하나에 순서대로 쌓여 있었고, 일부 기능만 따로 재사용하기가 어려웠다.

셋째, 이미지 경로를 수집하는 로직이 get_image_paths()와 run_inference() 두 곳에 중복 구현되어 있었고, 둘 다 경로 전체를 list에 담아 반환하는 방식이었다.

3. 적용한 개선 방법

B. Generator 기반 이미지 경로 처리

기존의 get_image_paths()를 iter_image_paths() generator로 교체했다. os.walk()로 탐색한 경로를 list에 모으지 않고 yield로 하나씩 넘기도록 바꿨고, 배치 단위 처리를 위한 iter_batches()도 같은 방식으로 구현했다. 중복 구현되어 있던 경로 탐색 로직도 이 함수 하나로 통합했다.

C. 클래스 분리 및 단일 책임 원칙

run_experiment() 안에 뭉쳐 있던 역할을 세 클래스로 분리했다. ExperimentConfig는 dataclass로 설정값만 보유하고, YOLOTrainer는 학습/평가 실행만 담당하며, ResultLogger는 파일 저장과 로그 기록만 맡는다. 분리 후에는 실험 조건을 바꿀 때 Config 객체만 새로 만들어 넘기면 되고, Trainer나 Logger는 수정 없이 재사용할 수 있다.

D. 데코레이터 적용

반복적으로 등장하던 부가 로직 세 가지를 데코레이터로 분리했다. `@timer`는 `perf_counter`로 실행 시간을 측정해 (`result, elapsed`) 튜플로 반환하고, `@log_experiment`는 함수 시작/종료와 예외 발생을 logging으로 기록하며, `@validate_config`는 Config 값의 유효성을 사전에 검사한다. 셋 모두 `functools.wraps`를 적용해 원본 함수의 메타데이터를 보존했다.

4. 성능 측정 결과

측정 환경: Linux 5.15 (WSL2) / Python 3.8.10 / 10회 반복 측정 / synthetic 파일 트리 사용

이미지 수	Before (ms)	After (ms)	속도	Before 메모리	After 메모리
100장	0.271 ± 0.035	0.220 ± 0.013	1.23x	15.8 KB	14.4 KB
500장	1.009 ± 0.069	0.745 ± 0.017	1.35x	58.0 KB	58.8 KB
1000장	1.860 ± 0.061	1.433 ± 0.063	1.30x	115.4 KB	115.0 KB
5000장	8.866 ± 0.201	6.693 ± 0.179	1.32x	567.2 KB	565.0 KB

실행 시간은 모든 입력 크기에서 generator 방식이 약 1.23~1.35배 빠르게 측정됐다. After의 표준편차가 Before보다 작은 경우가 많아 측정 안정성도 높은 편이었다.

메모리는 입력 크기가 늘어도 두 방식 간 차이가 미미하고, 500장 구간에서는 after 쪽이 오히려 0.8KB 더 많이 나오는 역전 현상도 관찰됐다. 이는 `tracemalloc` 특성상 generator를 `list()`로 강제 소비하는 측정 방식에서 비롯된 노이즈로 판단된다. 실제 파이프라인에서 경로를 순차적으로 한 번만 소비하는 경우에는 `list`와 달리 전체 경로를 메모리에 유지하지 않으므로, 이미지 수에 비례한 메모리 절감 효과가 나타날 것으로 예상된다.

5. 결과 해석 및 한계

구조적 측면의 개선은 수치보다 명확했다. Before에서 모델이나 데이터셋을 바꾸려면 함수 내부로 들어가 수정해야 했지만, After에서는 `ExperimentConfig` 객체를 새로 정의하는 것만으로 새로운 실험을 구성할 수 있다. 데코레이터 도입으로 Trainer의 핵심 메서드에서 시간 측정, 로깅, 검증 코드가 모두 빠져나와 각 함수가 하나의 역할만 수행하게 됐다.

한계도 있다. generator는 한 번만 소비 가능해서, 동일한 경로 목록을 여러 번 순회해야 하는 경우에는 `list`가 낫다. 클래스 분리로 초기 코드량이 늘어나는 것도 단순한 1회성 실험에서는 불필요한 복잡도가 될 수 있다. 실제 YOLO 학습 환경이 없어 학습/평가 전후의 실행 시간을 직접 비교하지 못한 점은 이번 과제의 주요 한계다.